riskdep: DeFi risk dependencies

How the project works · DeFi Risk Dependency Challenge, Belem Capital, EPFL Hackathon 2026

The question we answer

- A dependency of a position is hit
  - How much is lost?
  - How fast does it land?
  - Stay or exit, given what exiting costs?
- Judged on: discovery 30 %, impact 30 %
  - decision usefulness 25 %, working demo on real data 15 %

Five modes, one command

Mode	For whom	What it gives
graph	anyone reviewing a position	the dependency graph, siblings, oracle classes
shock	vault owner, risk analyst	what each market and depositor loses when a node fails
analyze	risk or portfolio manager	loss, timing and STAY / REDUCE / EXIT
watch --vault	a Vault V2 depositor	alerts, block by block
watch --user	an address with Morpho positions	health, liquidation, rate alerts

The pipeline

- Discover: from one address to everything it depends on
- Price the risk: probabilities per layer, or a known incident
- Propagate: the shock engine, market by market
- Decide: verdict given the incident, breakeven p*
- Watch: detect it live, alert the depositor

1. Dependency discovery

Vault → adapters → Morpho Blue markets → oracles → feeds

Who controls each contract: proxy admins, owners, timelocks

Issuers and their backing (registry, Pharos)

Sibling markets sharing a collateral, asset or price input

Oracle class: price-tracking or constant-price

2–3. Impact: two engines

Shock engine (shock, analyze --vault)

- A failure on a node of the graph
- Each market re-evaluates every borrower
- Edge rules carry it to the next node
- Repeated to a fixed point

Monte Carlo (analyze, PT positions)

- A probability per layer, per year
- 20,000 simulated months
- True value vs oracle value
- Expected loss, CVaR 99 %

4. Decision

Verdict given the incident: EXIT or STAY, and why

Breakeven p*: how likely must it be for leaving to pay

- Cost of leaving (yield given up, or an exit cost) ÷ loss saved

A book of vaults: act first on the lowest p*

Outside view: Nexus cover prices, shown, never used

5. Monitoring: watch

Polls every block (12 s), 2 blocks behind the head

Alerts: info, warn, critical, in the terminal and JSON

- Bad debt, oracle moves, liquidations, utilisation
- Upgrades, owners, feeds, timelock queues, mints

Cursor saved last: a restart misses no block

Cross-checks: a second RPC, the API against the chain

Data sources, all public

Source	Used for
Ethereum RPC (publicnode)	all on-chain state, blocks, logs
Morpho API	finding positions, borrowers, vault roles
Pendle API	PT markets, implied yield, exit quotes
DefiLlama	hack record, protocols, token prices
Nexus, Pharos (optional)	outside prices, issuer grades, backing
Etherscan (optional key)	oracle source verification

Is the number defensible?

PT-reUSD, 2026-08-25	Predicted	Real
Oracle fall (USDC market)	-2.45 % (input: observed)	-2.45 %
Positions liquidated	13	13, same borrowers
Debt liquidatable / repaid	35.46M USDC	35.19M USDC
Liquidation incentive	927,975	920,781
Bad debt	0	0

Demos on real data

- Offline, from mainnet recordings: no network, no key
  - graph, analyze --vault, analyze --book, watch --replay
- Crisis on a mainnet fork: scripts/fork/demo.sh --short
  - Issuer markdown, timelocked upgrade, bad debt, in 2 min
- Sponsor's vault (Belem RWA EURCV) and its curator's book

Interfaces

Command line

- python3 -m riskdep <mode>
- HTML report and graph
- JSON for every output

API and web

- HTTP API (uvicorn, OpenAPI)
- React UI: vault, market, graph
- Impact and alert pages

How it is built

Design first: plans in plan/, numbered decisions

Pure rules and engine, I/O at the edges

No literal thresholds: registry.json, each with a basis

One cached fetcher, one clock: any live run can be a test

288 tests, all offline; ruff in pre-commit and CI

Limits

Ethereum only; Solana, Base and other chains not read

Probabilities are base rates, layers mostly independent

analyze covers Pendle PT collateral on Morpho Blue

Free RPC: short state window, no backfill

Issuer data (NAV, reserves) taken as published